

EPIC EVENTS

Architecture technique et sécurité

CRM en ligne de commande - Python, Peewee, SQLite, Typer, JWT, Sentry et Pytest

Tests automatisés	156
Échecs	0
Couverture globale	77%
Couverture JWT	96%

Projet OpenClassrooms P12 - Juillet 2026

Sommaire

1. Synthèse de validation
2. Vue d'ensemble de l'architecture
3. Schéma de la base de données
4. Authentification JWT et autorisation
5. Sécurité et supervision avec Sentry
6. Stratégie de tests et couverture
7. Installation et démonstration
8. Vérification finale du dépôt

1. Synthèse de validation

Epic Events met en œuvre une architecture en couches, une base relationnelle SQLite, une authentification JWT, une autorisation par rôle, propriété et affectation, le principe du moindre privilège, la supervision Sentry et une suite de tests automatisés.

- 156 tests réussis, 0 échec.
- 77 % de couverture globale ; 96 % pour src/core/jwt_auth.py.
- 4 tests unitaires, 143 tests d'intégration et 9 tests fonctionnels.
- Base de tests SQLite isolée en mémoire et fichiers JWT de test temporaires.
- Secrets, token local, base locale et rapports générés exclus du dépôt Git.
- Événement contrôlé de démonstration reçu dans Sentry.

Indicateur	Valeur
Tests	156
Échecs	0
Couverture globale	77%
Couverture JWT	96%

2. Vue d'ensemble de l'architecture

2.1 Objectif

Le CRM est utilisé par les départements gestion, commercial et support. L'architecture sépare l'interaction terminal, la session d'authentification, les règles métier, la persistance et la supervision.

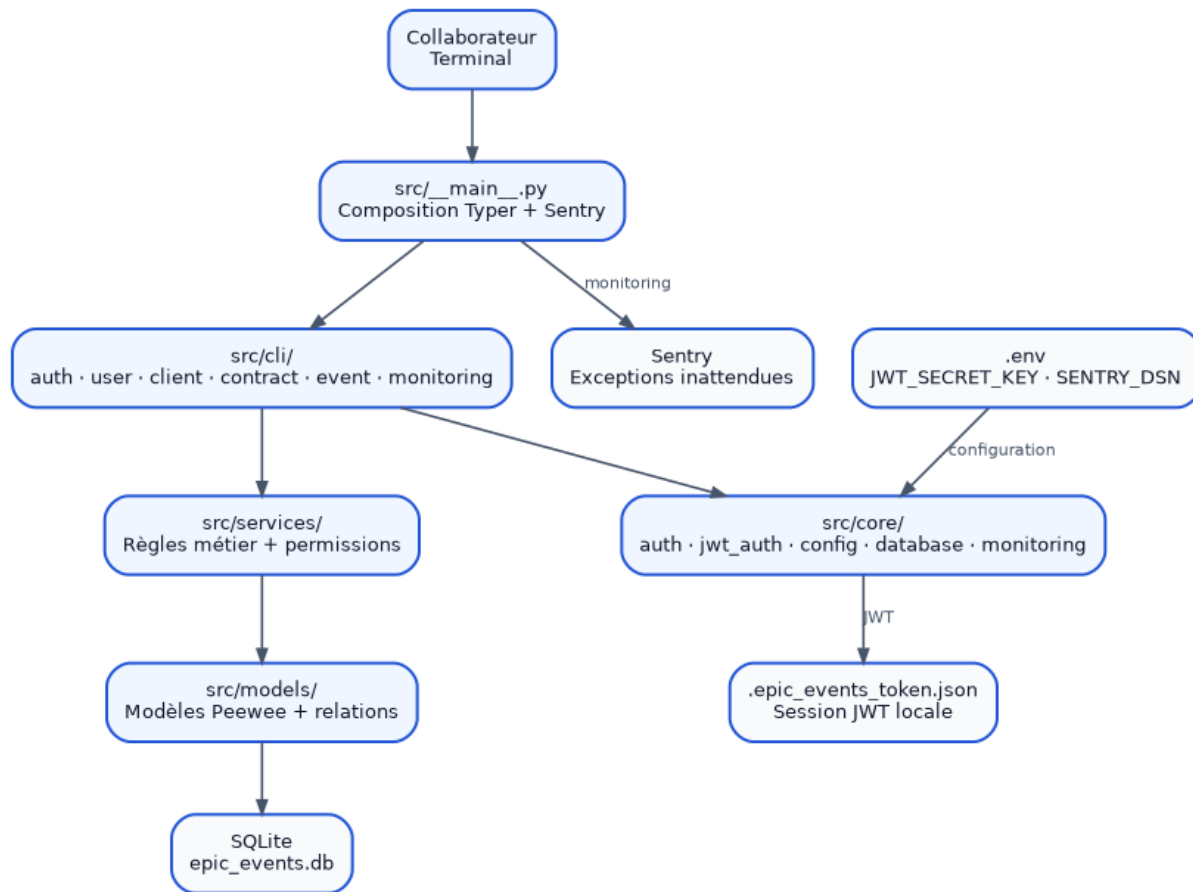


Figure 1 - Architecture en couches actuelle

2.2 Responsabilités des couches

Composant	Responsabilité
src/__main__.py	Compose les groupes Typer, initialise Sentry et capture les exceptions inattendues.
src/cli/	Collecte les arguments, affiche les résultats et convertit les erreurs métier en sorties CLI.
src/core/auth.py	Vérifie les identifiants bcrypt et fournit les contrôles génériques de département.
src/core/jwt_auth.py	Crée, signe, valide, charge, enregistre et supprime les JWT.
src/services/	Applique les permissions, la propriété, l'affectation et les validations métier.
src/models/	Définit le schéma Peewee et les relations.
src/utills/	Initialise la base et le premier compte de gestion.
Sentry	Reçoit les exceptions techniques inattendues et la démonstration contrôlée.

2.3 Structure du code

```
src/
├── __main__.py
├── cli/
│   ├── auth.py  client.py  common.py
│   ├── contract.py  event.py  monitoring.py  user.py
├── core/
│   ├── auth.py  config.py  database.py
│   ├── jwt_auth.py  monitoring.py
├── models/
├── services/
├── utils/
│   ├── create_db.py
│   └── create_user.py
```

Le point d'entrée ne contient plus les règles métier. Les commandes sont regroupées par domaine avec `app.add_typer` et l'application se lance avec `python -m src`.

2.4 Flux d'une commande protégée



Figure 2 - Authentification puis autorisation

3. Schéma de la base de données

3.1 Choix SQLite et Peewee

SQLite est adapté à une application locale en ligne de commande : aucune infrastructure serveur n'est requise et l'installation reste reproductible. Peewee fournit le schéma par modèles, les clés étrangères et les requêtes paramétrées.

- Clés étrangères activées avec le pragma foreign_keys.
- RESTRICT protège les clients, contrats et commerciaux référencés.
- SET NULL permet de conserver un événement si son support n'est plus référencé.
- La désactivation logique des comptes préserve l'historique CRM.

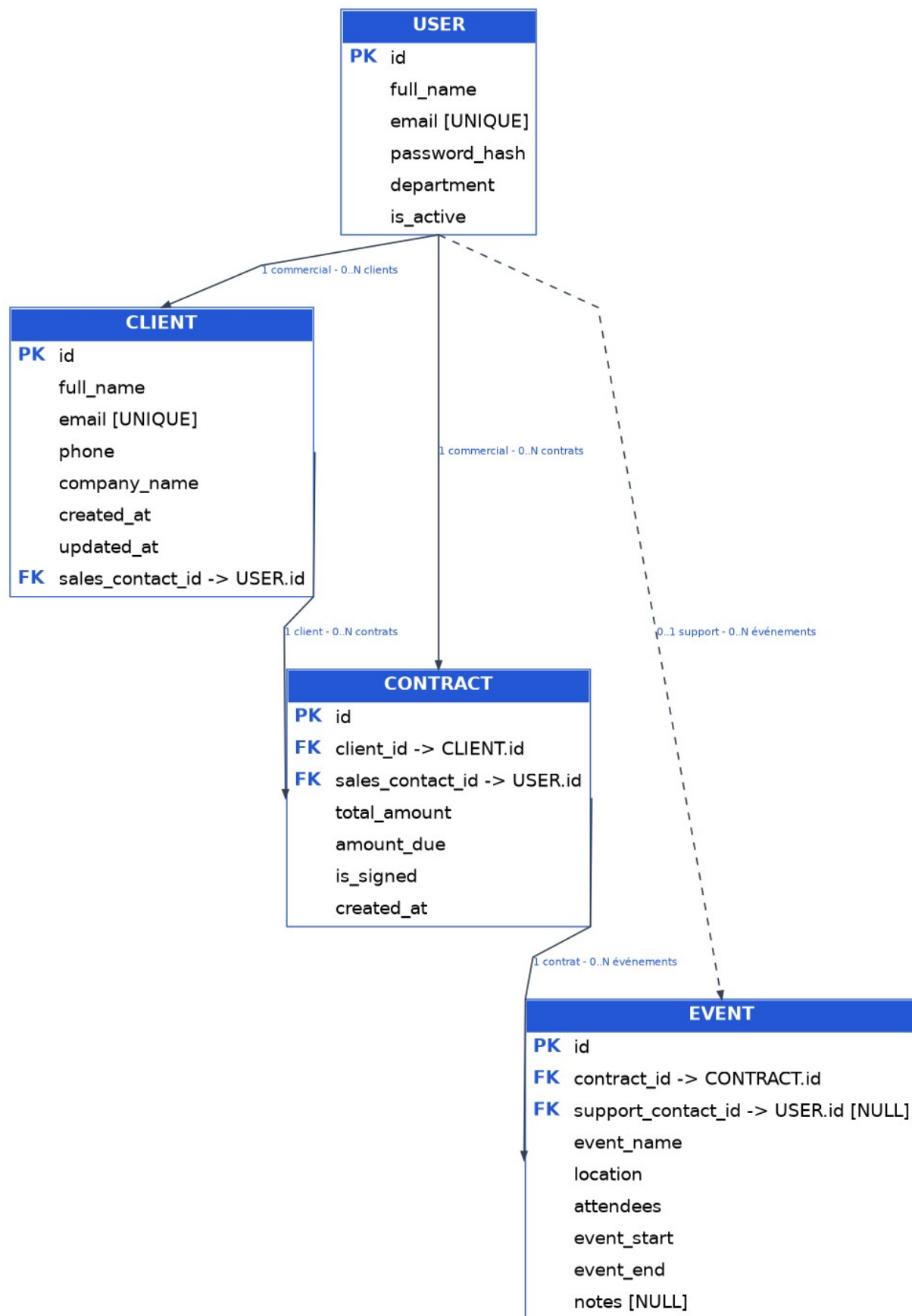


Figure 3 - Diagramme définitif des relations

3.2 Tables

users

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant du collaborateur
full_name	Varchar(255)	Obligatoire	Nom affiché
email	Varchar(255)	Unique, obligatoire	Identifiant de connexion
password_hash	Varchar(255)	Obligatoire	Hash bcrypt
department	Varchar(50)	Obligatoire	MANAGEMENT, COMMERCIAL ou SUPPORT
is_active	Boolean	Vrai par défaut	Statut logique

clients

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant du client
full_name	Varchar(255)	Obligatoire	Nom
email	Varchar(255)	Unique, obligatoire	Email
phone	Varchar(30)	Obligatoire	Téléphone
company_name	Varchar(255)	Obligatoire	Entreprise
created_at	Datetime	Automatique	Premier contact
updated_at	Datetime	Mis à jour	Dernier contact
sales_contact_id	Integer	FK users, RESTRICT	Commercial responsable

contracts

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant du contrat
client_id	Integer	FK clients, RESTRICT	Client
sales_contact_id	Integer	FK users, RESTRICT	Contact commercial
total_amount	Decimal(10,2)	Strictement positif	Montant total
amount_due	Decimal(10,2)	Entre 0 et total	Reste à payer
is_signed	Boolean	Faux par défaut	Signature
created_at	Datetime	Automatique	Création

events

Champ	Type	Contrainte	Rôle
id	Integer	Clé primaire	Identifiant de l'événement
contract_id	Integer	FK contracts, RESTRICT	Contrat
support_contact_id	Integer	FK users nullable, SET NULL	Support
event_name	Varchar(255)	Obligatoire	Nom
location	Varchar(255)	Obligatoire	Lieu
attendees	Integer	Supérieur à zéro	Participants
event_start	Datetime	Obligatoire	Début
event_end	Datetime	Après le début	Fin
notes	Text	Facultatif	Notes

3.3 Logique des relations

- Un commercial peut gérer plusieurs clients et contrats.
- Un client peut posséder plusieurs contrats.
- Le contrat conserve le contact commercial associé au client.
- Un contrat peut couvrir plusieurs événements.
- Un événement peut être sans support ou avoir un seul support ; un support peut gérer plusieurs événements.

4. Authentification JWT et autorisation

4.1 Connexion et session

Le collaborateur saisit son email dans la commande auth login et son mot de passe dans un prompt masqué. Après vérification bcrypt, l'application crée un JWT signé avec HS256, valable 60 minutes, puis l'enregistre dans .epic_events_token.json.

```
JWT payload
{
  "sub": "<user_id>",
  "email": "<user_email>",
  "department": "<department>",
  "iat": "<issued_at>",
  "exp": "<expiration>"
}
```

Le token est un justificatif de session, pas la source finale des permissions. À chaque commande protégée, l'utilisateur est rechargé depuis SQLite et son compte doit toujours exister et être actif.

4.2 Différence entre authentification et autorisation

Concept	Question	Implémentation
Authentification	Qui est le collaborateur ?	bcrypt + JWT + rechargement SQLite
Autorisation	A-t-il le droit ?	require_permission + propriété + affectation dans les services

4.3 Matrice des permissions

Opération	Gestion	Commercial	Support
Lire clients, contrats et événements	Oui	Oui	Oui
Créer / modifier / désactiver un collaborateur	Oui	Non	Non
Créer un client	Non	Oui	Non
Modifier un client	Non	Ses clients	Non
Créer un contrat	Oui	Non	Non
Modifier un contrat	Tous	Clients propres	Non
Filtrer contrats non signés / non payés	Non	Oui	Non
Créer un événement	Non	Client propre + contrat signé	Non
Lister les événements sans support	Oui	Non	Non
Affecter un support	Oui	Non	Non
Lister / modifier ses événements	Non	Non	Affectations propres
Déclencher le test Sentry	Oui	Non	Non

4.4 Désactivation et sessions existantes

La suppression fonctionnelle d'un collaborateur est une désactivation logique. Les données liées sont conservées. Même si un token non expiré existe encore, le rechargement SQLite refuse immédiatement un compte désactivé ou supprimé.

5. Sécurité et supervision avec Sentry

Risque	Mesure
Mot de passe exposé	Prompt masqué, hash bcrypt, aucune valeur dans l'historique de commande.
Token falsifié ou expiré	Signature HS256, vérification exp et rejet des JWT invalides.
Compte révoqué	Rechargement du User et contrôle is_active pour chaque commande.
Injection SQL	Requêtes Peewee paramétrées, jamais de SQL concaténé avec les entrées.
Élévation de privilège	Autorisations dans les services selon rôle, propriété et affectation.
Secret committé	Exclusion de .env, JWT_SECRET_KEY, token, base locale et rapports.
Fuite par monitoring	send_default_pii=False et include_local_variables=False.

5.1 Configuration des secrets

```
.env
JWT_SECRET_KEY=<long random secret>
SENTRY_DSN=<optional Sentry DSN>
SENTRY_ENVIRONMENT=development
```

La clé JWT et le DSN réel ne sont jamais stockés dans le code. .env et .epic_events_token.json sont ignorés par Git.

5.2 Architecture Sentry

```
src/__main__.py -> initialize_sentry()

Expected business error
-> explicit CLI message

Unexpected technical exception
-> capture_application_exception()
-> Sentry event ID

Controlled demonstration
python -m src monitoring test-sentry
```

La commande de démonstration est réservée à la gestion. Sans DSN, elle s'arrête avec un message explicite ; avec un DSN valide, elle envoie une RuntimeError contrôlée et retourne l'identifiant Sentry.

5.3 Limites acceptées du JWT local

- Le token est un bearer token local : le système d'exploitation doit protéger le fichier.
- Il n'existe pas de liste de révocation centrale ni de refresh token.
- Ces limites sont adaptées au périmètre d'un CRM CLI local avec expiration courte et revalidation SQLite.

6. Stratégie de tests et couverture

6.1 Organisation

```
tests/
├── unit/
│   └── test_monitoring.py
├── integration/
│   ├── test_auth.py
│   ├── test_client_service.py
│   ├── test_contract_service.py
│   ├── test_database.py
│   ├── test_event_service.py
│   ├── test_jwt_auth.py
│   └── test_user_service.py
└── functional/
    └── test_cli.py
```

Catégorie	Périmètre	Résultat
Unitaires	Monitoring isolé avec mocks	4
Intégration	Auth, JWT, services, modèles, SQLite mémoire	143
Fonctionnels	CLI publique avec CliRunner	9
Total	Suite complète	156 / 0 échec

6.2 Couverture validée

Coverage report: 77%
156 tests passed - 0 failures - src/core/jwt_auth.py: 96%

File	statements	missing	coverage
src/__main__.py	27	10	63%
src/cli/auth.py	28	11	61%
src/cli/client.py	35	13	63%
src/cli/common.py	23	6	74%
src/cli/contract.py	56	40	29%
src/cli/event.py	76	46	39%
src/cli/monitoring.py	21	2	90%
src/cli/user.py	32	22	31%
src/core/auth.py	31	1	97%
src/core/config.py	18	1	94%
src/core/database.py	3	0	100%
src/core/jwt_auth.py	52	2	96%
src/core/monitoring.py	20	1	95%
src/models/*	57	0	100%
src/services/client_service.py	50	0	100%
src/services/contract_service.py	71	0	100%
src/services/event_service.py	97	3	97%
src/services/user_service.py	68	0	100%
src/utils/*	27	27	0%
TOTAL	792	185	77%

coverage.py v7.15.2 - validated on Python 3.12.2 - July 2026

Figure 4 - Rapport de couverture actualisé

Composant	Couverture
Modèles	100%
Service clients	100%
Service contrats	100%
Service utilisateurs	100%
Service événements	97%
Authentification bcrypt	97%
Moteur JWT	96%
Monitoring	95%
Total	77%

Les scripts utilitaires d'initialisation restent à 0 % car ils sont exécutés comme outils d'installation ; les services métier et les modules de sécurité conservent une couverture élevée.

6.3 Commandes de validation

```
python -m pytest -q
python -m pytest tests/unit -v
python -m pytest tests/integration -v
python -m pytest tests/functional -v
python -m pytest --cov=src --cov-report=term-missing --cov-report=html
```

7. Installation et démonstration

7.1 Installation dans un environnement vierge

```
git clone https://github.com/AlNocquet/OC-Projet12-EpicEvents.git
cd OC-Projet12-EpicEvents
python -m venv venv
.\venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
Copy-Item .env.example .env
python -c "import secrets; print(secrets.token_urlsafe(48))"
# Copier la clé générée dans JWT_SECRET_KEY
python -m src.utils.create_db
python -m src.utils.create_user "Morgan Manager" manager@epicevents.com
python -m src --help
```

7.2 Parcours d'authentification

```
python -m src.auth.login manager@epicevents.com
python -m src.auth.check-management
python -m src.client.list
python -m src.auth.logout
```

Après la connexion, les commandes métier n'exigent plus l'email ni le mot de passe. Le token expire après 60 minutes ou disparaît lors du logout.

7.3 Proposition de démonstration en dix minutes

Étape	Démonstration	Point expliqué
1	Architecture et schéma	Packages, responsabilités, PK/FK, SQLite et Peewee.
2	Connexion JWT	Prompt masqué, bcrypt, token local, expiration et logout.
3	Gestion	Création d'un collaborateur, contrat et affectation support.
4	Commercial	Création client, filtres, contrat propre et événement signé.
5	Support	Liste et mise à jour des événements affectés.
6	Sécurité	Moindre privilège, paramètres Peewee, secrets et désactivation.
7	Tests	156 tests, 77 % global, 96 % JWT.
8	Sentry	Exception contrôlée visible dans le tableau de bord.

8. Vérification finale du dépôt

```
git status --short
git log --oneline --decorate --graph --all
git ls-files .env .epic_events_token.json epic_events.db
python -m pytest -q
python -m pytest --cov=src --cov-report=term-missing
```

- Arbre de travail propre.
- Aucun secret, token JWT ou base locale suivi par Git.
- README français et anglais reproductibles.
- Schéma relationnel, ADR, journaux et UML présents.
- Suite complète et couverture reproductibles.

Conclusion

L'architecture actuelle répond aux exigences fonctionnelles, techniques et de sécurité du CRM. Elle fournit une session JWT utilisable, une autorisation toujours fondée sur l'état SQLite, des permissions de moindre privilège, une supervision Sentry, un schéma relationnel documenté et une validation automatisée de 156 tests.